
Data Fetching & API Integration Lab (Practical)

Welcome to the practical lab for Data Fetching & API Integration! In this session, you will install **Axios**, implement asynchronous API communication inside your React application, and handle loading and error states while preparing your frontend to communicate with your future FastAPI backend.

1. Concept Overview

In this lab, you will:

- Install and configure **Axios** for handling HTTP requests.
 - Manage asynchronous request lifecycle states: **Loading**, **Success**, and **Error**.
 - Simulate sending data to an API endpoint and rendering the returned response dynamically.
-

2. Step-by-Step Lab Instructions

Step 1: Install Axios

Open your terminal in your project directory (`fullstack-m1-ui`) and install Axios:

```
npm install axios
```

Step 2: Update `PredictionForm.jsx` to Include Asynchronous Fetching

Open `src/components/PredictionForm.jsx` and update it to simulate an API request using `async/await`, complete with loading spinners and error handling. For now, we will use a public mock API endpoint (like `[https://jsonplaceholder.typicode.com/posts]` (`https://jsonplaceholder.typicode.com/posts`)) to test network communication before your FastAPI server is ready.

```
// src/components/PredictionForm.jsx
import { useState } from "react";
```

```

import axios from "axios";
import "./PredictionForm.css";

function PredictionForm() {
  const [feature1, setFeature1] = useState("");
  const [feature2, setFeature2] = useState("");
  const [predictionResult, setPredictionResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setError(null);
    setPredictionResult(null);

    try {
      // Simulating a POST request to a backend endpoint
      // Later, this will point to: "http://127.0.0.1:8000/predict"
      const response = await
    axios.post("https://jsonplaceholder.typicode.com/posts", {
      feature1: Number(feature1),
      feature2: Number(feature2),
    });

    // Simulate receiving a prediction score back from the server
    // (Using dummy logic for now: sum of inputs * 1.5)
    const simulatedPrediction = (Number(feature1) + Number(feature2)) *
1.5;

    setPredictionResult({
      prediction: simulatedPrediction,
      status: "Success",
      serverMessage: response.statusText,
    });
  } catch (err) {
    setError("Failed to connect to the backend server. Please try
again.");
    console.error("API Error:", err);
  } finally {
    setLoading(false);
  }
};

return (
  <main className="form-container">
    <section className="card">
      <h2>Model Feature Input</h2>
      <form onSubmit={handleSubmit}>
        <div className="input-group">
          <label htmlFor="feature1">Primary Feature (e.g., Sq Ft /
Hours):</label>

```

```

        <input
          type="number"
          id="feature1"
          value={feature1}
          onChange={(e) => setFeature1(e.target.value)}
          placeholder="Enter value..."
          required
        />
      </div>

      <div className="input-group">
        <label htmlFor="feature2">Secondary Feature (e.g., Rooms /
Age):</label>
        <input
          type="number"
          id="feature2"
          value={feature2}
          onChange={(e) => setFeature2(e.target.value)}
          placeholder="Enter value..."
          required
        />
      </div>

      <button type="submit" className="submit-btn" disabled={loading}>
        {loading ? "Processing Inference..." : "Run Prediction"}
      </button>
    </form>

    {/* Error State Display */}
    {error && <p className="error-message">{error}</p>}}

    {/* Success Result Display */}
    {predictionResult && (
      <div className="result-box">
        <h3>Prediction Result:</h3>
        <p>Estimated Output: <strong>{predictionResult.prediction}
</strong></p>
        <p className="status-success">Status: Connected & Processed
Successfully</p>
      </div>
    )}
  </section>
</main>
);
}

export default PredictionForm;

```

Step 3: Add Loading and Error Styles to `PredictionForm.css`

Open `src/components/PredictionForm.css` and append styling rules for loading states and error messages:

```
/* Add to bottom of src/components/PredictionForm.css */
.submit-btn:disabled {
  background-color: #94a3b8;
  cursor: not-allowed;
}

.error-message {
  margin-top: 1rem;
  padding: 0.75rem;
  background-color: #fffeed;
  color: #dc2626;
  border-radius: 6px;
  font-size: 0.875rem;
  border-left: 4px solid #dc2626;
}

.status-success {
  margin-top: 0.75rem !important;
  font-weight: 600;
  color: #16a34a !important;
  font-size: 0.85rem !important;
}
```

Step 4: Test Your Asynchronous Component

1. Run your development server (`npm run dev`).
 2. Enter values in your form and click **Run Prediction**. Notice the button text change to **Processing Inference...**, followed by the successful display of your calculated result!
-

3. Reference Links & Further Reading

- **[Axios POST Requests Guide](#)**: Learn how to structure payloads and headers for REST APIs.
 - **[React Managing State for Async Actions](#)**: Best practices for tracking loading, error, and data states.
-